LeRobot (Free Pascal) User Manual

9 November 2026

LeRobot is a drawing robot you steer on screen: either live, with the mouse and keyboard (Dessiner), or by writing a short Pascal program of robot commands (such as `Dessin_du_Robot.p`). This version compiles the Turbo Pascal programs of the 1991 book *Premières leçons de programmation en Turbo Pascal* (Macintosh) with Free Pascal, unchanged, and runs them in the robot window of the OCaml and Python versions, with the same behaviour. It lives in the folder `LeRobot_FreePascal_FR` of the `LeRobot` folder, which can be anywhere (commands below are typed in a Terminal window opened in the `LeRobot` folder); the installation is described in the *LeRobot (Free Pascal) Installation Manual* (`LeRobot_Install.pdf`).

Contents

1 Getting started **1**

2 Part 1 — Interactive drawing (Dessiner) **2**
2.1 Launch . . . 2
2.2 The window . . . 2
2.3 Moving the robot with the mouse or the arrow keys . . . 2
2.4 Keyboard commands . . . 3
2.5 Finishing and saving . . . 3

3 Part 2 — Running a robot program **3**
3.1 Run Dessin_du_Robot.p . . . 3
3.2 Write your own program . . . 4
3.3 Errors . . . 4
3.4 Programs of the book . . . 5
3.5 Replaying at another speed . . . 5
3.6 Command reference . . . 5

4 Troubleshooting **7**

1 Getting started

You need Free Pascal (`fpc`), a Python linked to Tk 8.6 or later (for example the one from python.org), and the folders `LeRobot_FreePascal_FR` and `LeRobot_Python_FR` side by side (see the installation manual). There is no XQuartz: the robot window is an ordinary macOS window.

There are two ways to start a program:

- **The application** `LeRobotFreePascal.app` (in `LeRobot_FreePascal_FR`): double-click it to run Dessiner; drop a `.p` file on its icon to compile and run that program.

- **The script** `robot.sh`, in a Terminal window opened in `LeRobot_FreePascal_FR`: `./robot.sh path/to/Program.p`. The program can be in any folder.

Both compile the program with Free Pascal (the compiler messages appear only if there is an error), then run it. The compiled programs are kept in the folder `_build`, which can be deleted at any time.

How it works. The Macintosh of 1991 drew with QuickDraw, which does not exist any more. The units `Robot1`, `Robot2` and `Robot3` of this version send each command of the program (`av`, `lg(12)`, ...) to the robot window of the Python version, which draws it and answers. The program itself is an ordinary compiled Pascal program: `write`, `writeln` and `readln` use the Terminal window.

The robot window is 800 × 700 points. The large white area is where the robot draws; the strip along the bottom is the message bar.

2 Part 1 — Interactive drawing (Dessiner)

In Dessiner you give the robot commands with single keys and move it with the mouse or the arrow keys; press Enter when you are done, then `y` to save the drawing as a Pascal program.

2.1 Launch

Double-click `LeRobotFreePascal.app`, or:

```
cd LeRobot_FreePascal_FR
./robot.sh livre/Didactitiels_et_jeux/Dessiner.p
```

`Dessiner.p` is the program of 1991: it calls the procedure `interaction` of the robot with the drawing and saving options. The robot appears in the centre, pointing north. The window comes to the front with the keyboard focus; if keys do nothing, click in the window.

2.2 The window

- **Drawing area** — the large white area.
- **Message bar** (bottom) — a reminder of the keys, ending with `* :cc w :peindre Entree :fin`.
- **Colour box** (small robot near the right of the bar) — shows the current pen colour.
- **Enter icon** (far right) — the Enter symbol; it blinks when the robot waits for a key.

2.3 Moving the robot with the mouse or the arrow keys

Click on a grid point (or move the pointer with the button held down) and the robot jumps to the nearest grid point, without drawing. The arrow keys move it one grid step left, right, up or down, also without drawing. When the pointer moves with the button released, the robot does not move.

If the robot would go past the edge of the drawing area, it stays where it is and the terminal prints `** SORTIE INTERDITE ** translater` (“forbidden exit”), with a beep-beep. This is only a warning.

2.4 Keyboard commands

The robot draws only when its pen is down (`b`; it is down at the start). One step is one grid square.

Key	Command	What it does
`b`	bc	Pen down: the robot draws as it moves
`l`	lc	Pen up: the robot moves without drawing
`a`	av	Move forward one step
`)`	vd	Curved turn to the right (draws an arc)
`(`	vg	Curved turn to the left (draws an arc)
`+`	pqtd	Quarter turn right (90°)
`-`	pqtg	Quarter turn left (90°)
`d`	phtd	Eighth turn right (45°)
`g`	phtg	Eighth turn left (45°)
`c`	dc	Draw a small cross (+)
`p`	dp	Draw a point
`.`	dg	Draw the grid points
`*`	cc	Next pen colour (shown in the colour box)
`w`	peindre	Paint the closed area behind the robot with the pen colour
Arrow keys	translater	Move one step left, right, up or down, without drawing
Backspace	—	Undo the last command
Enter	—	Finish the drawing

Colours cycle in this order: white, yellow, blue, violet, green, red, indigo, black. To paint, put the robot so that its back is inside a closed area, then press `w`.

2.5 Finishing and saving

1. Press **Enter**. The bar asks: *Voulez-vous sauver dans .../Dessin_du_Robot.p ? (y/n)*
2. Press `y` to save. Every command you used, including mouse and arrow moves (written as `translater(dx, dy)`) and colour changes (written as `cc(n)` with the colour name in a comment), is written to `Dessin_du_Robot.p` as a Pascal program (see Part 2), and the window closes. The Terminal prints the command to replay it.
3. Press any other key to close without saving.

Saving always overwrites `Dessin_du_Robot.p`, in the `LeRobot_FreePascal_FR` folder. To keep an earlier drawing, copy that file under another name first.

3 Part 2 — Running a robot program

A robot program is a Pascal program that uses the unit `Robot1` and lists robot commands in order. It draws the picture by itself, then waits so you can replay it at another speed or close it with Esc.

3.1 Run Dessin_du_Robot.p

```
cd LeRobot_FreePascal_FR && ./robot.sh Dessin_du_Robot.p
```

or drop `Dessin_du_Robot.p` on `LeRobotFreePascal.app`. A saved drawing looks like this:

```
{$mode macpas}
program Dessin_du_Robot;
 uses Robot1, Robot2;
 { Mon joli dessin, engendre par Dessiner }
begin
 ce;
 av; av; pqtd; av; av; cc(2); { bleu }
 av; translater(5, 3); av; vg; av; st;
end.
```

3.2 Write your own program

Create a file anywhere, for example `Maison.p`:

```
program Maison;
 uses Robot1;
 { Dessiner un carre }
begin
 av; av; pqtd; av; av; pqtd;
 av; av; pqtd; av; av; pqtd;
 st;
end.
```

Run it with `./robot.sh path/to/Maison.p` from the `LeRobot_FreePascal_FR` folder, or drop it on `LeRobotFreePascal.app`.

Every program follows the same pattern:

- `uses Robot1;` after the `program` line. All the commands are in `Robot1`; `Robot2` and `Robot3` (sounds, texts, input in 1991) are empty and may be listed too, as in the programs of the book. The window opens at the first command, with the robot in the centre, pointing north, pen down, in black, with a grid of 12 points.
- The robot starts with its pen down, so every move draws: call `lc` to move without drawing, and `bc` to draw again.
- `st` last: shows the speed screen and keeps the window open. Without it, the bar says *Fin du programme : taper une touche ou cliquer* and the window closes at the next key or click.
- Commands without arguments are written without parentheses (`av`, not `av()`); commands are separated by `;`. Pascal does not distinguish upper and lower case: `AV`, `Av` and `av` are the same command.
- The program may be written in UTF-8 (accents allowed in strings and comments) or come from the Macintosh of 1991 (MacRoman encoding, Mac units such as `Memtypes`, `Quickdraw`): `robot.sh` converts the latter first, into `_build/convertis`, without changing the original. The line numbers of the error messages stay those of the original.

3.3 Errors

- **Compilation errors** are found by Free Pascal before the program runs; the window does not open. The message gives the line and column:

```
Maison.p(5,2) Error: Identifier not found "avv"
Maison.p(5,2) Error: Wrong number of parameters specified for call to "av"
```

- **Leaving the window**: when the robot would go past the edge, the program stops with a beep-beep and the Terminal prints `Erreur : le robot sortirait du cadre (av)`; the window stays open to show the drawing made so far.

3.4 Programs of the book

The folder `livre` holds the 407 programs of the book, one subfolder per chapter (the same names as in the OCaml and Python versions):

```
./robot.sh livre/20_grille_polaire/LimaconDePascal.p
./verifier.sh                          # all programs, one after the other
./verifier.sh 02_interaction           # one chapter
./verifier.sh '' FriseDeTriangles      # resume at a program
```

With `verifier.sh`, answer in the robot window when a drawing is finished: Enter = correct (next program), `r` = run it again, `q` or Esc = stop (the command to resume is printed), any other key starts a remark, ended by Enter, noted in `verification.txt`. Compilation errors are noted there too (`Cocotte-errone.p` has one on purpose, as in the book). A program that never ends by itself (`Boucle1`, `Boucle2`, `MontreDigitale`) is stopped with Esc. Programs that ask questions (`readln`) ask them in the Terminal: click in the Terminal window to answer. `LEROBOT_MUET=1` turns the sound off.

The programs are the Macintosh programs of 1991, converted once by `outils/convertir.py` (UTF-8, units of the robot, directive `{$mode macpas}` for the Macintosh dialect of Pascal); their text is unchanged. Some of them were changed later in the OCaml and Python editions of the book, so a few drawings differ from those versions.

3.5 Replaying at another speed

When the program reaches `st`, the bar shows:

```
+ : plus vite   - : moins vite   vitesse : N     autre touche : redessiner   Echap :
    quitter
```

- `+` / `-` — change the speed from 0 to 10 (default 10, the fastest). Only the number changes.
- Any other key, for example the space bar — clear the window and redraw at the chosen speed; the speed screen comes back when it is done.
- Esc — close the window.

At speed 1 each command takes about 0.1 s. Speed 0 is step-by-step mode: the robot does one command each time you press a key. While the robot draws, Esc (or `§` or `#`) stops the program.

3.6 Command reference

One step is one grid square. Commands are in French; the table gives the English meaning. The numbers in brackets are parameters (`integer` or `real`); `var` parameters receive a result.

Command	Meaning
`av`	Move forward one step
`avf(n)`	Move forward n steps (backwards if $n < 0$)
`vd` / `vg`	Curved turn right / left (draws an arc)
`phtd` / `phtg`	Turn 45° right / left
`pqtd` / `pqtg`	Turn 90° right / left

Command	Meaning
`p3htd` / `p3htg`	Turn 135° right / left
`pdt`	Turn around (180°)
`pn pne pe pse ps psw pw pnw`	Face N, NE, E, SE, S, SW, W, NW
`pivoter(n)`	Face direction n (`nord` = 0, `nordEst` = 1 ... `nordOuest` = 7)
`rt(a)`	Turn by a degrees (anticlockwise), keeping the step length
`bc` / `lc`	Pen down / pen up
`dp` / `dc`	Draw a point (a square of side `ec`) / a small cross
`cc(n)`	Pen colour n (`blanc` = 0, `jaune`, `bleu`, `violet`, `vert`, `rouge`, `indigo`, `noir` = 7)
`ec(n)`	Pen thickness n (a square pen, as on the Macintosh)
`cf(n)`	Clear the window and fill it with background colour n
`peindre`	Paint the closed area behind the robot with the pen colour
`translater(dx, dy)`	Move dx steps across and dy steps up, without drawing
`tracer(x, y)`	Go to window pixel (x, y), drawing if the pen is down
`deplacer(x, y)`	Go to grid point (x, y), counted from the centre
`tl(l)`	Move forward l points, the grid following the robot
`ce`	Go to the centre
`eh eb ad ag`	Go to the top, bottom, right or left edge
`dg`	Draw the grid points
`lg(x)` / `lgX(x)` / `lgY(x)`	Set the step size (both directions / across / up)
`lgRT(r, t)`	Step of length r at angle t (polar grid)
`montrerRobot` / `cacherRobot`	Show / hide the robot
`autoriserSorties` / `interdireSorties`	Allow / forbid moving past the window edge
`vt(n)`	Set the speed (0 to 10)
`pp` / `ex`	Step-by-step mode / full speed
`message('text')`	Write a message in the bottom bar
`ecrire('text')`	Write a text at the robot's position
`entierEnChaine(n)`, `reelEnChaine(x)`	A number as a string, for `message` and `ecrire`
`lireCar(c)`	Wait for a key in the robot window; its character is put in `c` (a `var` parameter of type `char`)
`interaction(x, y, c, ...)`	Let the user move the robot (see `Dessiner.p`)
`note(h, d)`, `silence(d)`, `bip`	Play a note, a rest, a beep
`delai(t)`	Wait t seconds
`heure(h, m, s)`, `date(j, m, a)`	The time, the date (`var` parameters)
`write`, `writeln`, `readln`	Write and read in the Terminal (standard Pascal)
`st`	End: show the speed screen

Test functions such as `tb` (would the next step leave the window?) and the values `valX`, `valLgX`, `valOrientation`, ... are listed with all the other commands, constants (colours, directions, notes) and their parameter types in the interface of `unites/Robot1.pas`. The architecture and the conversion of the programs are described in the `README.md` file of `LeRobot_FreePascal_FR`.

4 Troubleshooting

Symptom	Cause	Fix
Prog.p(5,2) Error: Identifier not found "avv"	A command misspelled (or a missing `uses Robot1;`)	Correct line 5, column 2
Wrong number of parameters specified for call to "av"	`av()` or `av(3)` written for `av`	Write `av` without parentheses, or `avf(3)`
Erreur : le robot sortirait du cadre (av)	The robot was asked to move past the window edge	Change the program, or call `autoriserSorties`
`** SORTIE INTERDITE` `** translater` in the terminal	In Dessiner, a move past the window edge	Only a warning
The program never ends	An endless loop (`while true`, `repeat ... until false`, endless recursion)	Press Esc in the robot window
The program waits and the window shows nothing new	It asks a question with `readln` in the Terminal	Click in the Terminal window and answer
Keys do nothing in the window	The window does not have keyboard focus	Click in the window
Digits are not accepted	French AZERTY keyboard: the top row gives & é " ... without Shift	For `lireCar`, use Shift or the numeric keypad
Ce programme doit etre lance par robot.sh	The compiled program was started by itself	Use `./robot.sh Prog.p` or `LeRobotFreePascal.app`
A saved drawing was lost	Saving from Dessiner always overwrites `Dessin_du_Robot.p`	Copy it under a new name before saving again
Text font cannot be changed	`policeTexte` and `styleTexte` have no effect	None; texts use Helvetica 13